

TAREA 2: INTRODUCCION A LA CRYPTOGRAFIA

Layme Rodas Daniel Leoncio

Ing. Ciberseguridad

Ejercicio 1: Pares e impares

ASCII asigna a cada carácter un número entero entre 0 y 127. Al guardar una bandera se cometió un error: los caracteres que ocupaban una posición par se almacenaron en una lista y los que ocupaban una posición impar en otra.

Reconstruya la cadena original intercalando ambas listas y obtenga la bandera.

pares = [99, 121, 116, 123, 114, 95, 95, 104, 95, 111, 95, 110, 101, 115, 115]

impares = [114, 112, 111, 111, 100, 121, 99, 114, 115, 110, 105, 118, 114, 111, 125]

```
def ejercicio1():
    pares = [99, 121, 116, 123, 114, 95, 95, 104, 95, 111, 95, 110, 101, 115, 115]
    impares = [114, 112, 111, 111, 100, 121, 99, 114, 115, 110, 105, 118, 114, 111, 125]
    flagASCII = []
    for i in range(15):
        flagASCII.append(pares[i])
        flagASCII.append(impares[i])
    print(flagASCII)
    flag = ""
    for i in range(len(flagASCII)):
        flag += chr(flagASCII[i])
    print(flag)
```

Script:

Resultado:

```
flagASCII = [99, 114, 121, 112, 116, 111, 123, 111, 114, 100, 95, 121, 95, 99, 104, 114, 95, 115, 111, 110, 95, 105, 110, 118, 101, 114, 115, 111, 115, 125]
```

```
flag= crypto{ord_y_chr_son_inversos}
```

Ejercicio 2: Hexadecimal al revés

La siguiente cadena hexadecimal contiene los bytes de una bandera, pero fueron escritos en orden inverso: el último byte del mensaje aparece primero.

7d73657665725f6c615f6e61765f73657479625f736f6c7b6f7470797263

Decodifique la cadena a bytes, restablezca el orden correcto y obtenga la bandera.

```
def ejercicio2():
    cadena = "7d73657665725f6c615f6e61765f73657479625f736f6c7b6f7470797263"
    cadenaBYTES = bytes.fromhex(cadena)
    flag = cadenaBYTES[::-1]
    print(flag)
```

Script:

Resultado

```
flag = b'crypto{los_bytes_van_al_reves}'
```

Ejercicio 3: De Base64 a hexadecimal

Tome la siguiente cadena Base64, decodifíquela a bytes y entregue dos resultados: la representación hexadecimal de esos bytes y la bandera en texto.

Y3J5cHRve2RlX2Jhc2U2NF9hX2hleGFkZWNPbWFSfQ==

Script:

```
def ejercicio3():
    cadenaB64 = "Y3J5cHRve2RlX2Jhc2U2NF9hX2hleGFkZWNPbWFSfQ=="
    cadenaBytes = base64.b64decode(cadenaB64)
    cadenaHex = cadenaBytes.hex()
    print(cadenaBytes)
    print(cadenaHex)
```

Resultado:

cadenaBytes = b'crypto{de_base64_a_hexadecimal}'

cadenaHex = 63727970746f7b64655f6261736536345f615f68657861646563696d616c7d

Ejercicio 4: Dos capas

Una bandera se codificó primero en Base64. La cadena Base64 resultante, que es texto ASCII, se volvió a codificar en hexadecimal. Solo se conserva el resultado final:

59334a3563485276653252766331396a595842686331396b5a56396a623252705a6d6c6a59
574e7062323539

Deshaga las dos capas en el orden correcto y recupere la bandera. Indique también cuál era la cadena Base64 intermedia.

Script:

```
def ejercicio4():
    cadena_hex = "59334a3563485276653252766331396a595842686331396b5a56396a623252705a6d6c6a59574e7062323539"
    base64_bytes = bytes.fromhex(cadena_hex)
    base64_intermedia = base64_bytes.decode('ascii')

    flag_bytes = base64.b64decode(base64_intermedia)

    print("Cadena Base64 intermedia:", base64_intermedia)
    print("Flag:", flag_bytes.decode())
```

Resultado:

Cadena Base64 intermedia: Y3J5cHRve2Rvc19jYXBhc19kZV9jb2RpZmljYWNPb259

Flag: crypto{dos_capas_de_codificacion}

Ejercicio 5: Dos mitades enteras

Una bandera de longitud par se partió exactamente por la mitad. Cada mitad se convirtió a un número entero tomando sus bytes ordinales, concatenándolos en hexadecimal e interpretando el resultado en base 10. Ambas mitades tienen la misma longitud en bytes.

mitad_1 = 132187956505730919844267377848747253607

mitad_2 = 152037883334028404060266402501018743677

Convierta cada entero nuevamente en bytes, concaténalos en orden y obtenga la bandera.

Script:

```
def ejercicio5():
    mitad1_int = 132187956505730919844267377848747253607
    mitad2_int = 152037883334028404060266402501018743677

    bytes1 = long_to_bytes(mitad1_int)
    bytes2 = long_to_bytes(mitad2_int)

    flag_bytes = bytes1 + bytes2
    print(flag_bytes.decode())
```

Resultado:

crypto{enteros_grandes_partidos}

Ejercicio 6: Máscara en Base64

Una bandera fue cifrada mediante XOR con una máscara aleatoria de la misma longitud. El texto cifrado se publicó en hexadecimal y la máscara, por comodidad, en Base64.

cifrado = 1f13cd301580c8ed672d94581eb381011c8fb7e9b94c23e25b

mascara = fGG0QGHvs4AGXvc5bNLeZHLQ1YjKKRXWJg==

Lleve ambos valores a bytes, aplique XOR byte a byte y obtenga la bandera.

```
def ejercicio6():
    cifrado_hex = "1f13cd301580c8ed672d94581eb381011c8fb7e9b94c23e25b"
    mascara_b64 = "fGG0QGHvs4AGXvc5bNLeZHLQ1YjKKRXWJg=="

    cifrado_bytes = bytes.fromhex(cifrado_hex)
    mascara_bytes = base64.b64decode(mascara_b64)

    flag_bytes = bytes([c ^ m for c, m in zip(cifrado_bytes, mascara_bytes)])
    print(flag_bytes.decode())
```

Script:

Resultado:

crypto{mascara_en_base64}

Ejercicio 7: Cada byte con su índice

En este cifrado no hay una clave fija: cada byte del mensaje fue XORado con su propia posición dentro de la cadena (el primer byte con 0, el segundo con 1, el tercero con 2, y así sucesivamente).

63737b73706a7d64696d6b546e747a6a4f727d7d4b66634871777e727f7863

Recupere la bandera.

Script:

```
def ejercicio7():
    cifrado_hex = "63737b73706a7d64696d6b546e747a6a4f727d7d4b66634871777e727f7863"
    cifrado_bytes = bytes.fromhex(cifrado_hex)

    flag_bytes = bytes([b ^ i for i, b in enumerate(cifrado_bytes)])
    print(flag_bytes.decode())
```

Resultado:

crypto{cada_byte_con_su_indice}

Ejercicio 8: Cadena de cuatro claves

Se generaron cuatro claves aleatorias de la misma longitud que la bandera. A partir de ellas se publicaron los siguientes valores en hexadecimal:

CLAVE1 = 8dfd58871a293c61ba91ba7a2db4afaeb10b396552738b9813eda2eb98f2ba

CLAVE1 ^ CLAVE2 =

5f991a2109f2627642f887cdf6356093294bdb263b60cb9389362cd9949905

CLAVE2 ^ CLAVE3 =

e4645fe3f06489f439b462a56ff8d7532805cd8a0ceb1ca323b3036c2380c4

CLAVE3 ^ CLAVE4 =

e33b8767118894bb83e734dd2b6fbde1aa552701c05f553b127ebd8a66c8c7

BANDERA ^ CLAVE4 =

b649e3528658382830551ba6fa72c4eb7f6357adcbc468f7ce7851b02850c1

Usando únicamente las propiedades conmutativa, asociativa, de identidad y de autoinverso, determine qué combinación de las cuatro primeras líneas aísla CLAVE4 y recupere la bandera.

```
def ejercicio8():
    clave1 = bytes.fromhex("8dfd58871a293c61ba91ba7a2db4afaeb10b396552738b9813eda2eb98f2ba")
    k1_k2 = bytes.fromhex("5f991a2109f2627642f887cdf6356093294bdb263b60cb9389362cd9949905")
    k2_k3 = bytes.fromhex("e4645fe3f06489f439b462a56ff8d7532805cd8a0ceb1ca323b3036c2380c4")
    k3_k4 = bytes.fromhex("e33b8767118894bb83e734dd2b6fbde1aa552701c05f553b127ebd8a66c8c7")
    flag_k4 = bytes.fromhex("b649e3528658382830551ba6fa72c4eb7f6357adcbc468f7ce7851b02850c1")

    clave4 = bytes([a ^ b ^ c ^ d for a, b, c, d in zip(clave1, k1_k2, k2_k3, k3_k4)])

    flag_bytes = bytes([f_k4 ^ k4 for f_k4, k4 in zip(flag_k4, clave4)])
    print(flag_bytes.decode())
```

Script:

Resultado:

crypto{propiedades_encadenadas}

Ejercicio 9: Byte secreto sobre Base64

Una bandera se codificó en Base64 y la cadena resultante se cifró aplicando XOR con un solo byte secreto, el mismo para todos los caracteres. El resultado se publicó en hexadecimal:

046e17683e150f2b386f1768391a0b3b3e6f0b373e300b6d3f6c64273f6f1724070b64340405

133113370f64

No se conoce el byte utilizado. Pruebe los 256 valores posibles y decida cuál es el correcto.

Script:

```
def ejercicio9():
    cifrado_hex = "046e17683e150f2b386f1768391a0b3b3e6f0b373e300b6d3f6c64273f6f1724070b64340405133113370f64"
    cifrado_bytes = bytes.fromhex(cifrado_hex)

    for key in range(256):
        candidato_b64_bytes = bytes([b ^ key for b in cifrado_bytes])
        try:
            candidato_b64_str = candidato_b64_bytes.decode('ascii')
            decoded_bytes = base64.b64decode(candidato_b64_str)
            decoded_text = decoded_bytes.decode('ascii')

            if decoded_text.startswith("crypto{") and decoded_text.endswith("}"):
                print(f"Byte clave encontrado: {hex(key)}")
                print(f"Cadena Base64: {candidato_b64_str}")
                print(f"Flag: {decoded_text}")
                break
        except Exception:
            continue
```

Resultado:

Byte clave encontrado: 0x5d

Cadena Base64: Y3J5cHRve2J5dGVfc2VjcmV0b19zb2JyZV9iYXNlNjR9

Flag: crypto{byte_secreto_sobre_base64}

Ejercicio 10: Clave repetida

La bandera fue cifrada con XOR usando una clave corta que se repite cíclicamente a lo largo del mensaje. No se conoce ni la clave ni su longitud.

3b2b23272c36213434382c32072b3f273d2d3333390623083e36283a392d35083b3634383b303e3825

Recupere la clave y la bandera.

```
def ejercicio10():
    cifrado_hex = "3b2b23272c36213434382c32072b3f273d2d3333390623083e36283a392d35083b3634383b303e3825"
    cifrado_bytes = bytes.fromhex(cifrado_hex)

    prefix = b"crypto{"

    clave_parcial = bytes([c ^ p for c, p in zip(cifrado_bytes[:7], prefix)])

    clave_str = clave_parcial.decode('ascii')

    for l in range(1, len(clave_parcial)):
        if clave_str[:l] * (len(clave_str) // l) == clave_str[:l * (len(clave_str) // l)]:
            clave = clave_str[:l].encode('ascii')
            break
    else:
        clave = clave_parcial

    flag_bytes = bytes([c ^ clave[i % len(clave)] for i, c in enumerate(cifrado_bytes)])
    print(f"Clave recuperada: {clave.decode()}")
    print(f"Flag: {flag_bytes.decode()}")
```

Script:

Resultado:

Clave recuperada: XYZW

Flag: crypto{clave_repetida_y_formato_conocido}